

3. 佇列與堆疊

想到堆疊，想到深度；想到深度就想到歐陽修這首蝶戀花：

「庭院深深深幾許，楊柳堆煙，簾幕無重數。玉勒雕鞍遊冶處，樓高不見章臺路。
雨橫風狂三月暮，門掩黃昏，無計留春住。淚眼問花花不語，亂紅飛過鞦韆去。」

關於窗戶：

「Where there's will, there's a way;
where there's way, there's a wall;
where there's wall, there's a window.」

這一章介紹三個基本的資料結構：佇列 (queue)、堆疊 (stack) 與雙向佇列 (deque，念做 "deck"，正式名稱是 double-ended queue)，除了說明基本原理與應用外，也介紹一種與雙向佇列類似的演算法：滑動視窗 (sliding window)。另外我們也介紹鏈結串列 (linked list)，這是一種在實際用途很廣，但是在解題上比較少用的資料結構。這幾種資料結構都是課本上的基本資料結構，有很多應用。滑動視窗這個名字倒是有很多不同的稱呼方式，也有稱為爬行法，或認為是雙指針 (two-pointers) 方法的一種。反正名稱不是那麼重要，懂得原理與會運用才是最重要的事。我們先介紹基本原理與實作方式，然後再來看應用的題目。

3.1. 基本原理與實作

佇列、堆疊與雙向佇列都是簡單的資料結構，簡單到可以自己實作，但也有 STL 的容器可以使用，自己實作的在碰到空間問題時會比較麻煩，所以使用 STL 或許比較好一點。不過在解題的場合，很少會碰到空間問題，因此兩種方式都可以。這三種資料結構都是 "線性" 的，可以在一維陣列實作。鏈結串列最大的優點是將線性的資料放在不連續的位置，這特性在實際系統與軟體中有時非常重要，也因如此，串列需要以指標與記憶體配置來搭配。但解題的場合很少有空間的問題，也幾乎都避免使用 pointer，所以串列部分我們只說明解題中常見的方法。

佇列 (queue)

佇列就好像是一群排队排成一列，有一個出口與一個入口，通常出口稱為前端 (front) 而入口稱謂後端 (back)。成員進入時只能從入口進入，也就是加入尾端；離

開時只能依序從出口離開，佇列的特性是先進先出 (FIFO, first-in first-out)。與陣列一樣，在標準的佇列中，所有的成員必須是相同的資料型態。佇列通常需要四個基本操作，新增、刪除、查看出口的成員、以及檢查是否為空，有時也會查詢目前成員數。

我們可以很簡單的以陣列來實作佇列，在解題的場合，空間通常不是問題，所以我們宣告一個足夠大的陣列，初始時後端在陣列開頭的位置，每次加入時，後端往後移，每次刪除時，從陣列前端離開，在一連串操作時，佇列相當於在此陣列中往後滑動。我們需要維護好兩個變數，front 記著佇列前端出口位置，back 記著佇列後端入口位置。請看下面的範例程式展示著自製佇列的基本用法以及支援的四個操作。我們沿用 STL 的名詞，加入稱為 push，刪除稱為 pop，這名字其實來自堆疊。

```
// self-made queue
#include <bits/stdc++.h>
#define N 1000
int main() {
    int Q[N], front=0, back=0;
    int n, t;
    char cmd[10];
    scanf("%d", &n);
    for (int i=0; i<n; i++) {
        scanf("%s", cmd);
        if (strcmp(cmd, "push")==0) {
            scanf("%d", &t);
            Q[back++]=t; // push(t)
        }
        else if (strcmp(cmd, "pop")==0) {
            if (front==back) printf("empty\n");
            else {
                t=Q[front++]; // pop to t
                // save Q[front] to t, and then front++
                printf("%d\n", t);
            }
        }
    }
    return 0;
}
```

這個程式在檔案中有輸入資料可以測試，它有若干個命令，每個命令是 push 一個數字或是 pop，程式讀取每一個命令，如果是 push 就將資料加入 queue，如果是 pop 要先檢查佇列是否是空的，否則刪除資料並輸出。下面是以 STL 提供的 queue 來做一樣的事，目的只是展示如何操作。宣告時 queue<int> Q，意義就是宣告一個 int 型態的 queue 名字叫做 Q。

```
// ch3_2 STL queue
```

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    queue<int> Q;
    int n, t;
    char cmd[10];
    scanf("%d", &n);
    for (int i=0;i<n;i++) {
        scanf("%s", cmd);
        if (strcmp(cmd,"push")==0) {
            scanf("%d", &t);
            Q.push(t);
        }
        else if (strcmp(cmd,"pop")==0) {
            if (Q.empty()) printf("empty\n");
            else {
                t=Q.front();
                Q.pop();
                printf("%d\n",t);
            }
        }
    }
    return 0;
}

```

堆疊(Stack)

堆疊像是把子彈放入子彈夾(可能沒看過)，或者想像把硬幣堆疊在桌上。堆疊的出口與入口相同，就是最上面那一個，通常稱為 top。成員進入時只能從上方壓入(往上疊)，稱為 push；離開時只有頂端 top 的成員可以離開。堆疊的特性是後進先出(LIFO, last-in first-out)。在標準的堆疊中，所有的成員也必須是相同的資料型態。堆疊通常需要四個基本操作，push、pop、查看 top 的成員、以及檢查是否為空。

以陣列來實作堆疊比佇列更簡單，因為只有一端開口，我們將堆疊的底部設在陣列開頭之處，每次 push 時 top 往後移，每次 pop 時，top 往前移，在一連串操作時，堆疊只是 top 位置前後移動，像是一端黏在牆壁的彈簧，我們只要維護好一個變數 top 就可以了。請看下面的範例程式，它的測試資料與佇列相同。不熟悉的讀者可以用範例程式來了解用法。

```

// ch3_3 self-made stack, test data = ch3_1.in
#include <bits/stdc++.h>
#define N 1000
int main() {
    int S[N], top=-1;

```

```

int n, t;
char cmd[10];
scanf("%d", &n);
for (int i=0;i<n;i++) {
    scanf("%s", cmd);
    if (strcmp(cmd,"push")==0) {
        scanf("%d", &t);
        S[++top]=t; // push(t)
    }
    else if (strcmp(cmd,"pop")==0) {
        if (top<0) printf("empty\n");
        else {
            t=S[top--];
            printf("%d\n",t);
        }
    }
}
return 0;
}

```

下面的範例是以 STL 提供的 stack 來做一樣的事。宣告時 `stack<int> S`，意義就是宣告一個 `int` 型態的 stack 名字叫做 `S`。

```

// ch3_4 STL stack, test data at ch3_1.in
#include <bits/stdc++.h>
using namespace std;
#define N 1000
int main() {
    stack<int> S;
    int n, t;
    char cmd[10];
    scanf("%d", &n);
    for (int i=0;i<n;i++) {
        scanf("%s", cmd);
        if (strcmp(cmd,"push")==0) {
            scanf("%d", &t);
            S.push(t); // push(t)
        }
        else if (strcmp(cmd,"pop")==0) {
            if (S.empty()) printf("empty\n");
            else {
                t=S.top();
                S.pop();
                printf("%d\n",t);
            }
        }
    }
    return 0;
}

```

雙向佇列 (deque)

如果了解了 queue 的操作之後，deque 的操作就容易了解了，它與 queue 很類似，差異在它的前後都可以進出，所以加入與刪除的操作有 `push_front`, `push_back`, `pop_front`, `pop_back` 四種，這些操作的意義都可以從名字上了解。雙向佇列建議直接使用 STL，因為自己做的話，會需要處理空間的問題，比較麻煩，除非使用時已知它只會從一端加入，那就跟 queue 的製作很類似了。因為與 queue 很類似，這裡我們就不再提供範例，在後面的題目中會碰到。

3.2. 應用例題與習題

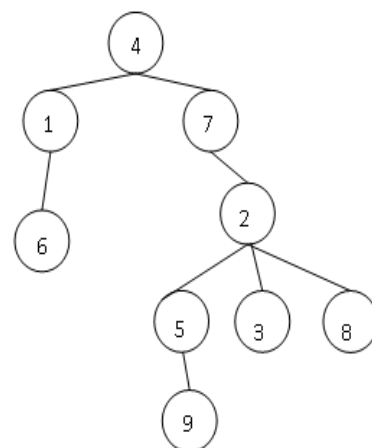
佇列堆疊與雙向佇列常常是用其他演算法裡面做為一個基本步驟，單獨出現的不算太多。尤其單純 queue 的題目並不多，除非是簡單的操作模擬，但是在樹與圖的走訪卻是非常重要的，這些要等到後續單元介紹了圖與樹的基本概念後再介紹比較合適。這裡提出一個例題，事實上是 tree 的題目，因為不需要太多專業術語，可以先放在此處當作 queue 的例題，等到後面的章節會再提出其他解法。

P-3-1. 樹的高度與根 (bottom-up) (APCS201710)

有一個大家族有 N 個成員，編號 $1 \sim N$ ，我們請每個成員寫下此家族中哪幾位是他的孩子。我們要計算每個人有幾代的子孫，這個值稱為他在這個族譜中的高度，也就是說，編號 i 的人如果高度記做 $h(i)$ ，那麼 $h(i)$ 就表示在所有 i 的子孫中，最遠的子孫與他隔了幾代。本題假設除了最高的一位祖先（稱為 root）之外，其他成員的 parent 都在此家族中（且只有一個 parent）。本題要計算所有成員高度的總和以及根的編號。

右圖是一個例子，每個成員是一個圈起來的號碼，劃線相聯的表示上面的點是下面點的 parent。編號 4 是根，有兩個孩子 1 與 7。編號 6, 9, 3, 8 都沒有孩子， $h(6)=h(9)=h(3)=h(8)=0$ ，此外 $h(2)=2$ 因為 9 與他隔兩代。你可以看出來 $h(5)=h(1)=1$, $h(7)=3$, $h(4)=4$ ，所以高度總和是 11。

Time limit: 1 秒



輸入格式：第一行有一個正整數 N 。接下來有 N 行，第 i 行的第一個數字 k 代表 i 有 k 個孩子，第 i 行接下來的 k 個數字就是孩子的編號。每一行的相鄰數字間以空白隔開。 $N \leq 1e5$ 。

輸出：第一行輸出根的編號，第二行輸出高度總和。

範例輸入：

```
9
1 6
3 5 3 8
0
2 1 7
1 9
0
1 2
0
0
```

範例結果：

```
4
11
```

沒有處理過樹狀圖的人可能會覺得這題複雜，但其實不難，這裡我們盡量不使用樹的術語來講。首先我們用個陣列把每個人的 `parent` 讀進來，第一個問題是如何找根。沒有 `parent` 的就是根，所以很簡單，只要看看哪個點沒有 `parent` 就行了。第二個問題要找高度，因為我們有每個點的 `parent`，一個直覺的做法就是：

從每一個點 i 出發，依據 `parent` 一直往上走直到無路可走（到達 `root`），並且用一個記步器 `cnt` 記錄走了幾步。每到一個點 j ，根據 `cnt` 就可以知道 i 是 j 的第幾代子孫。因為高度是要取所有子孫離他最遠的，所以到達 j 點時，就取 $h[j] = \max(h[j], cnt)$ ；在所有的點都走完之後，各點的高度就已經算好了。

夠簡單吧！以下是這個解法的範例程式。

```
// tree height slow method O(nL), L=tree height
#include <bits/stdc++.h>
using namespace std;
#define N 100010

int main() {
    int parent[N]={0}; // parent of node i
    int h[N]={0}; // height of node i
```

```

int deg[N]; // num of children
int n, i, j, ch;
scanf("%d", &n);
for (i=1; i<=n; i++) {
    scanf("%d", &deg[i]); // num of children
    for (j=0; j<deg[i]; j++) {
        scanf("%d", &ch);
        parent[ch] = i; // set parent
    }
}
int root;
for (root=1; root<=n; root++)
    if (parent[root]==0) break; // no parent
printf("%d\n", root);
long long step=0;
for (i=1; i<=n; i++) { // for each
    int cnt=0; // num of step from i
    // go up until root
    for (j=i; j!=0; j=parent[j]) {
        h[j]=max(h[j],cnt);
        cnt++; step++;
    }
}
long long total=0;
for (i=1; i<=n; i++)
    total += h[i];
printf("%lld\n",total);
fprintf(stderr," %lld steps, ans=%lld\n", step, total);
return 0;
}

```

這程式雖然容易理解，答案跑出來也是對的，但是有個問題，跑太慢，他的複雜度是多少呢？以一個極端的例子來看，如果這個家族 10 萬個成員全部都是單傳，最高祖先的高度是 99999，那麼這個程式的複雜度是多少呢？你或許會抗議哪裡有這麼多代的家族呀！唉呀，競賽程式就是這樣會給刁鑽的測資，worst case 就真的很 worst，況且題目根本沒說是人的家族，細菌 10 萬代就很有可能。好了，廢話少說，因為上面這個程式裡，每個點都要往上爬到根，所以在這個極端的例子裏，最下面的要跑 99999，倒數第二的要跑 99998，...，所以總共的複雜度約是 1 加到 99999，複雜度是 $O(n^2)$ ，一秒是跑不完的。

找到問題就設法改善吧。程式跑太慢通常是做了太多重複的事，在前面這個 10 萬代單傳的例子裏，這個方法之所以慢，就是走了太多一樣的路：最下一層的往上爬了 N 層，他的 parent (倒數第二層) 爬了 $N-1$ 層，兩位爬的路幾乎重複，假設倒數第二層確定知道自己的高度是 1 的話，那這兩位重疊的路只要爬一次不需要爬兩次。也就是說，對於 i 點，如果 $h[i]$ 已經確定，那麼 i 的子孫不必每位都爬 i 到根那段路，只要從 i 的高度往上計算爬一次就好了。那麼，要如何確定 i 的高度呢？根據定義，只要 i 的子孫都已經爬到 i 點， i 的高度就確知了。因為子孫關係有遞移性，我們得到一個結論：

如果我們可以調整計算的順序，使得「每個點都在他的所有孩子之後計算，那麼，每個點只需要向他的 parent 回報自己的高度就可以了」。

這樣一個順序我們稱為由下而上的順序(bottom-up sequence)。假設 seq[] 存放著一個 bottom-up 順序，那麼，計算高度只需要以下簡單的程式碼：

```
for (i=0; i<n; i++){
    int v = seq[i];
    h[parent[v]] = max(h[parent[v]], h[v]+1);
}
```

所以重點在於如何找出一個 bottom-up 順序，這就是 queue 出場的時候了，請看以下的算法。我們說一個點可以離開是指他的孩子都已經離開，顯然這樣的離開順序是一個 bottom-up sequence：

對所有 i ，以 $\text{deg}[i]$ 紀錄 i 目前還在的孩子數量，以一個 queue Q 放置所有可以離開但尚未離開的點，一開始把所有 $\text{deg}[]$ 為 0 的點放入 Q 中。每次從 Q 中拿出一個點 v ，讓 v 離開，更新他 parent 的 $\text{deg}[]$ ，如果他 parent 的 $\text{deg}[]$ 降為 0，就將他 parent 放入 Q 。

結合這樣的順序，我們可以將上面的程式改成下面的範例程式，這裡我們使用 STL 的 queue，另外我們也不需要特別去找 root，因為 bottom-up 順序的最後一個必然是 root。

```
// tree height bottom-up dp, using STL queue
#include <bits/stdc++.h>
using namespace std;
#define N 100010

int main() {
    int parent[N]={0}; // parent of node i
    int h[N]={0}; // height of node i
    int deg[N]; // num of children
    queue<int> Q;
    int n, i, j, ch;
    scanf("%d", &n);
    for (i=1; i<=n; i++) {
        scanf("%d", &deg[i]);
        if (deg[i] == 0) Q.push(i);
        for (j=0; j<deg[i]; j++) {
            scanf("%d", &ch);
            parent[ch] = i;
        }
    }
    int root, total=0; // total height
    // a bottom-up traversal
    while (1) { // queue is not empty
        int v = Q.front(); // pop Q to v
```



```

    Q.pop();
    total += h[v]; // add to total
    int p = parent[v];
    if (p == 0) { // root
        root = v;
        break;
    }
    h[p] = max(h[p], h[v]+1);
    if (--deg[p] == 0) // push parent to queue
        Q.push(p);
}
// root must be the last one in the queue
printf("%d\n%d\n", root, total);
return 0;
}

```

計算一下時間複雜度，輸入把 $\text{deg}[]$ 為 0 的放入 Q 中，花了 $O(n)$ ；while 迴圈裡面只有 $O(1)$ 的運算，問題是 while 迴圈做了幾次，因為每次有一點離開點 Q ，而一個點不可能進入 Q 兩次，所以整個時間複雜度是 $O(n)$ ，比改善前好太多了。這個題目在 APCS 五級分的程度已經算難的，如果第一次看 tree 就看得懂，恭喜你，如果不太能了解，沒關係，後面還有專門的章節來講樹狀圖。

接下來要看個比較簡單的例子，這是堆疊運用的經典題目。

P-3-2. 括弧配對

在本題中我們假設有三種括弧： $\{\}$ ， $()$ ， $[]$ 。輸入一個由括弧組成的字串，請判斷是否是**平衡的**，括弧可以巢狀但是不可以交叉，例如「 $() [] \{ [] \}$ 」、「 $[(()) \{ }]$ 」、「 $[() \{ } \{ } \} \{ } ()$ 」都是平衡的，但「 $([])$ 」不是平衡的，此外「 $() []$ 」也不是，因為[沒有配對。

Time limit: 1 秒

輸入格式：輸入包括若干行，最多 20 行，每行是一個表示式，由六個括弧字元組成的字串，沒有其他字元，字串長度不超過 150。

輸出：依序輸出每行是否是平衡的括弧，是則輸出 yes，否則輸出 no。

範例輸入：

```

() [] { [] }
[ ( ( ) ) { } ]
[ ( ) { } { } ] { } ( )
( [ ] )

```

() [

範例結果：

```
yes
yes
yes
no
no
```

括弧配對其實是計算表示式 (expression evaluation) 的其中一個環節，是資料結構課程中的一個重要課題，因為這是程式 compiler 裡面一定要做的工作。括弧的結構事實上也是遞迴定義的，所以可以用遞迴的方式做，但用堆疊更簡單。用堆疊檢查括弧是否平衡的算法也很直覺，基本上我們需要找到每一個右括弧對應的左括弧，算法如下：

啟用一個 stack *S*。掃描輸入字串的每一個字元，

如果是左括弧的一種：將他壓入 *S*；

否則 (是右括弧)，取出堆疊最上方的左括弧與其配對，如果不配，則錯誤。

如果字串結束後堆疊中還有東西，也是錯誤。如果都沒錯誤則是平衡的。

本題有三種括弧，在實作時要避免很繁瑣的窮舉三種括弧，我們使用了一個小技巧，請看以下的範例程式。將 6 種括弧字元對應到整數 0~5，先排左括弧再排右括弧，同類的一對的讓他們剛好差 3，這樣可以簡單的檢查是否配對。為了做這個對應，我們先宣告了一個字串 `ch[7]="([{}])"`；對每一個輸入字元 `in[i]`，用一個字串函數 `strchr()` 來找出 `in[i]` 在 `ch[]` 中的位置，這個函數如果不熟悉，也可以自己寫一個簡單的迴圈 (如同程式中的註解)。

```
// p3-2 parenthesis matching, no STL
#include <bits/stdc++.h>

int main() {
    char in[210], ch[7]="([{}])";
    int S[210], top; // stack
    while (scanf("%s", in) != EOF) {
        top = -1; // clear stack
        int len = strlen(in);
        assert(len <= 150);
        bool error = false;
        for (int i = 0; i < len; i++) {
            int k = strchr(ch, in[i]) - ch;
            //or using: for (k = 0; k < 6 && ch[k] != in[i]; k++);
            assert(k < 6); // no invalid char
            if (k < 3) // left
                S[++top] = k; // push
            else {
```

```

        if (top<0 || k!=S[top]+3) { //mismatch
            error=true;
            break;
        }
        top--; // pop
    }
}
if (top>=0) error=true;
printf((error)?"no\n":"yes\n");
}
return 0;
}

```

上面的程式是自製堆疊，如果使用 STL，則如下面的範例，程式雷同，就不多做解釋了。但是有件事情要提醒一下，因為這題有多筆測資要處理，你可以注意到我們把堆疊宣告在一個副程式中，每一筆測資呼叫一次副程式，這樣做的好處是每一筆測資進來的時候不必把 stack 清空，如果是寫在主程式中以迴圈處理每一筆測資(像上面那支範例的寫法)，那要留意你的寫法是否 stack 可能留有前一筆測資留下來的東西，而要將 stack 清空必須把他 pop() 到空為止。

```

// p3-2 parenthesis matching, using STL
#include <bits/stdc++.h>
using namespace std;

int sol() {
    stack<int> S;
    char in[210], ch[7]="([{}])";
    if (scanf("%s",in)==EOF) return -1;
    // fprintf(stderr,"input length=%d\n",strlen(in));
    bool error=false;
    for (int i=0,len=strlen(in);i<len;i++) {
        int sym=strchr(ch,in[i])-ch;
        if (sym>=6) {
            fprintf(stderr,"i=%d, %d, %c",i, sym, in[i]);
            return -1;
        }
        assert(sym>=0 && sym<6);
        if (sym<3) { // left
            S.push(sym);
        }
        else {
            if (S.empty() || sym!=S.top()+3) { //mismatch
                error=true;
                break;
            }
            //match
            S.pop();
        }
    }
}

```

```

    }
}
if (!S.empty()) error=true;
printf((error)?"no\n":"yes\n");
return 0;
}
int main() {
    while (sol()==0) ;
    return 0;
}

```

下一個習題，我們是一個簡化版的計算表示式，輸入一個數學運算式，請計算出他的值，計算式中的運算只有加減乘除，其中除法是整數除法，運算的數字都只有一位正整數。

Q-3-3. 加減乘除

輸入一個只有加減乘除的計算式，請計算並輸出其結果。其中的數字皆為一位正整數，計算結果的絕對值不會超過 10 億。

Time limit: 1 秒

輸入格式：輸入一行是一個計算式，長度不超過 100。假設是一個正確的計算式。

輸出：輸出計算結果。

範例輸入：

5+2*3-6-7*2

範例結果：

-9

提示：因為加減乘除都是左結合的二元運算，左結合是指連續兩個運算時先算左邊，例如 $2-3-5=(2-3)-5$ 。現在需要考慮的只有先乘除後加減。

一開始先將第一個數字放起來，接著從前往後掃描計算式，每次考慮兩個字元，第一個一定是個運算符號 (op) 第二個一定是個數字 (num)。如果 op 是個乘法或除法，我們可以立刻將前面的運算結果與 num 做運算，因為沒有別的運算優先權高與乘除。如果是加法或減法，我們必須考慮兩種情形：(1) 如果之前有被保留的運算要先執行，目前的運算要保留，因為後面可能是乘除法；(2) 如果之前沒有被保留的運算，這個運算要被保留。在考慮一連串的運算過程中，最多只有一個運算會被保留，而且這個被保留的運算

必定是加法或減法。最後要提醒一點：全部的運算都考慮完畢後要檢查是否有被保留的運算。

完整的 expression evaluation 可以在資料結構的課本上或是網路上查到，要考慮的因素很多，所以做法比較複雜。

下面一題是堆疊運用的經典題。

P-3-4. 最接近的高人 (APCS201902, subtask)

對於一個人來說，只要比他高的人就是高人。有 N 個人排成一列，對於每一個人，要找出排在他前方離他最近的高人，排在之後的高人不算，假設任兩個相鄰的人的距離都是 1，本題要計算每個人和他前方最近高人的距離之總和。如果這個人前方沒有高人(前方沒人比他高)，距離以他的位置計算(等價於假設最前方有個無窮高的高人)。

Time limit: 0.5 秒

輸入格式：第一行有一個正整數 N ，第二行有 N 個正整數依序代表每個人的身高，相鄰數字間以空白隔開。 $N \leq 2e5$ ，身高不超過 $1e7$ 。

輸出：輸出每個人與前方最近高人的距離總和。

範例輸入：

```
8
8 6 3 3 1 5 8 1
```

範例結果：

```
18
```

為了方便起見，我們可以假設在最前方位置 0 的地方有個無限大的數字，這樣可以簡化減少邊界的檢查。最天真無邪又直接的方法就是：對每一個 i ，從 $i-1$ 開始往前一一尋找，直到碰到比他大的數字或者前方已走投無路。但是這個方法太慢，算一下複雜度，在最壞的情形下，每個人都要看過前方的所有人，所以複雜度是 $O(n^2)$ 。這一題 n 是 $2e5$ ，一定是 $O(n)$ 或 $O(n \log(n))$ 的方法。

要改善複雜度，要想想是否有沒有用的計算或資料。假設身高資料放在陣列 $a[]$ ，如果 $a[i-1] \leq a[i]$ ，那麼 $a[i-1]$ 不可能是 i 之後的人的高人，因為由後往前找的時候會先碰到 $a[i]$ ，如果 $a[i]$ 不夠高， $a[i-1]$ 也一定不夠高。同樣的道理，當我們計算到 i 的時候，任何 $j < i$ 且 $a[j] \leq a[i]$ 的 $a[j]$ 都是沒有用的。如果我們丟掉那些沒有用的，會剩下甚麼呢？一個遞減的序列，只要維護好這個遞減序列，就可以提高計算效率。單調序列要用二分搜嗎？其實連二分搜都不需要，想想看，當處理 $a[i]$ 時，我

們要先把單調序列後方不大於 $a[i]$ 的成員全部去除，然後最後一個就是 $a[i]$ 要找的高人，因為既然要做去除的動作，就一個一個比較就好了。那麼，要如何維護這個單調序列呢？第一個想法可能是將那些沒用的刪除，那可不行，陣列禁不起的折騰就是插入與刪除，因為必須搬動其後的所有資料。想想我們要做的動作：每次會從序列後方刪除一些東西，在加入一個東西。答案呼之欲出了，因為只從後方加入與刪除，堆疊是最適合的。請看以下的範例程式，這個程式非常簡單。

身高資料放在 $a[i]$ ，讀入之後就不再變動。準備一個堆疊 S 放置那個單調序列在 $a[]$ 中的索引位置。對每一個 $a[i]$ ，從堆疊中 pop 掉所有不大於它的東西，然後堆疊的最上方就是 i 的高人，接著把 $a[i]$ push 進堆疊。

```
// p_3_4a, stack for index
#include <bits/stdc++.h>
using namespace std;
#define N 300010
#define oo 100000001
int a[N];
stack<int> S; // for index
int main() {
    int i, n;
    long long total=0; // total distance
    scanf("%d", &n);
    S.push(0);
    a[0]=oo;
    for (i=1; i<=n; i++) {
        scanf("%d", &a[i]);
    }
    for (i=1; i<=n; i++) {
        while (a[S.top()] <= a[i])
            S.pop();
        total += i - S.top();
        S.push(i);
    }
    printf("%lld\n", total);
    return 0;
}
```

時間複雜度？看到雙迴圈以為是 $O(n^2)$ 嗎？跑跑看就知道這麼快一定不是 $O(n^2)$ ，難不成測資出太弱了！台語說：事情不像憨人想的那麼簡單。這個程式雖然有個雙迴圈，內層 while 迴圈也的確有可能某幾次跑到 $O(n)$ ，但是整體的複雜度其實只有 $O(n)$ ，原因是，雖然單一次 while 迴圈可能跑很多個 pop ，但是全體算起來不會 pop 超過 n 次，因為每個傢伙只會進入堆疊一次，進去一次不可能出來兩次，所以整體的複雜度是 $O(n)$ 。這種算總帳的方式來計算複雜度稱為均攤分析 (amortized analysis)，這個題目是均攤分析的經典教學題目。

上面的範例程式把資料放在 `a[]`，用 `stack` 存 `index`，事實上我們也可以只存那個單調序列，這樣在 `stack` 中要存身高與位置，以下是這樣的寫法，這只是給有興趣的人參考，上面那個寫法就夠好了。

```
// p_3_4, stack
#include <bits/stdc++.h>
using namespace std;
#define N 300010
#define oo 10000001
stack<pair<int,int>> S; // (b[i],i)
int main() {
    int i,bi,n;
    long long total=0; // total distance
    scanf("%d",&n);
    S.push({oo<<1,0});
    for (i=1; i<=n; i++) {
        scanf("%d",&bi);
        // <=bi is useless
        while (S.top().first <= bi)
            S.pop();
        total += i - S.top().second;
        S.push({bi,i});
    }
    printf("%lld\n",total);
    return 0;
}
```

前面的做法是由前往後掃，做到 `i` 的時候往前找 `i` 的對象（高人）。射鵰英雄傳的歐陽鋒都可以逆練九陰真經了，這個題目是不是可以倒著來解：由後往前，做到 `i` 時看看 `i` 是誰的對象（高人）。確實可以，基本想法是這樣：

由後往前掃，使用一個資料結構 `S` 存放目前尚未找到高人的人。掃到 `i` 時，我們檢查 `i` 是那些人的高人，也就是 `S` 中那些人的身高小於 `i` 的身高，然後將這些人從 `S` 中移除，最後把 `i` 放入 `S` 中，結束這一回合。

為了讓有興趣的人練習資料結構，我們介紹使用 `map` 來做的方法，因為身高可能相同，我們要用 `multimap`。（APCS 的難度應該不會考到 `map`，但有興趣的人或學習競賽的人可以練習。）

```
// p_3_4c, using multimap
#include <bits/stdc++.h>
using namespace std;
#define N 300010
#define oo 10000001
int a[N];
```

```

int main() {
    int i,n;
    long long total=0; // total distance
    scanf("%d",&n);
    a[0]=oo;
    for (i=1; i<=n; i++) {
        scanf("%d",&a[i]);
    }
    multimap<int,int> M; //(a[i],i) of unsolved
    for (i=n; i>=0; i--) {
        auto it=M.begin();
        // find the whose solution is i
        while (it!=M.end() && it->first<a[i]) {
            total += it->second - i; // add distance
            it=M.erase(it); // erase and point to next
        }
        M.insert({a[i],i});
    }
    printf("%lld\n",total);
    return 0;
}

```

下面這一題，是前一題的一種變化，我們當作習題。

Q-3-5. 帶著板凳排雞排的高人 (APCS201902)

身高高不一定比較厲害，個子矮的如果帶板凳可能會變高。有 n 個人排隊買雞排，由前往後從 1 到 n 編號，編號 i 的身高為 $h(i)$ 而他帶的板凳高度為 $p(i)$ 。若 j 在 i 之前且 j 的身高大於 i 站在自己板凳上的高度，也就是說 $h(j) > h(i) + p(i)$ ，則 j 是 i 的「無法超越的位置」，若 j 是 i 最後的無法超越位置，則兩人之間的人數 $(i-j-1)$ 稱為 i 的「最大可挑戰人數」 $S(i)$ ；如果 i 沒有無法超越位置，則最大可挑戰人數 $S(i) = i-1$ ，也就是排在他之前全部的人數。

舉例來說，假設 $n=5$ ，身高 h 依序為 $(5, 4, 1, 1, 3)$ 而板凳高度 p 依序是 $(0, 0, 4, 0, 1)$ 。計算可得 $h(i) + p(i) = (5, 4, 5, 1, 4)$ ，編號 1 的人前面沒有人，所以 1 的最大可挑戰人數 $S(1) = 0$ 。對編號 2 的人而言， $h(2) + p(2) = 4$ ，而往前看到第一個大於 4 的是 $h(1) = 5$ ，所以 $S(2) = 2-1-1 = 0$ 。而 $h(3) + p(3) = 5$ ，前面沒有人的身高大於 5，所以 $S(3) = 2$ 。而 $h(4) + p(4) = 1$ ， $h(2) = 4 > 1$ ，所以位置 2 是 4 的無法超越位置， $S(4) = 4-2-1 = 1$ 。同理可求出 $S(5) = 3$ ，他的不可超越位置是 1 的身高 5。

輸入 $h()$ 與 $p()$ ，請計算所有人 $S(i)$ 的總和。

Time limit: 1 秒

輸入格式：第一行為 n ， $n \leq 2e5$ ；第二行有 n 個正整數，依序是 $h(1) \sim h(n)$ ；第三行有 n 個非負整數，依序代表是 $p(1) \sim p(n)$ ；數值都不超過 $1e7$ ，同一行數字之間都是以空白間隔。

輸出：輸出 $S(i)$ 的總和。答案可能超過 2^{32} ，但不會超過 2^{60} 。

範例輸入：

```
5
5 4 1 1 3
0 0 4 0 1
```

範例結果：

```
6
```

Q-3-5 提示：本題與 P-3-4 的主要差異是多了板凳高度，另外有個小差異是計算的值略有不同。例題的題解中最重要的重點是我們只要維護一個單調遞減序列，既然是單調序列，就可以用二分搜找到第 i 個人站在板凳上的高度。此外，例題中我們介紹了另外一種使用 `multimap` 的解法，一樣可以稍作修改後解此題。

雖然保護樹木很重要，尤其學資料結構與演算法的人，對樹總是抱持著尊敬的心，但是有些樹種的目的就是要砍下來當木材的，請環保人士不要對下一題的名字抗議。來看下一個例題，砍樹。

P-3-6. 砍樹 (APCS202001)

N 棵樹種在一排，現階段砍樹必須符合以下的條件：「讓它向左或向右倒下，倒下時不會超過林場的左右範圍之外，也不會壓到其它尚未砍除的樹木。」。你的工作就是計算能砍除的樹木。若 $c[i]$ 代表第 i 棵樹的位置座標， $h[i]$ 代表高度。向左倒下壓到的範圍為 $[c[i]-h[i], c[i]]$ ，而向右倒下壓到的範圍為 $[c[i], c[i]+h[i]]$ 。如果倒下的範圍內有其它尚未砍除的樹就稱為壓到，剛好在端點不算壓到。

我們可以不斷找到滿足砍除條件的樹木，將它砍倒後移除，然後再去找下一棵可以砍除的樹木，直到沒有樹木可以砍為止。無論砍樹的順序為何，最後能砍除的樹木是相同的。

Time limit: 1 秒

輸入格式：第一行為正整數 N 以及一個正整數 L ，代表樹的數量與右邊界的座標；第二行有 N 個正整數代表這 N 棵樹的座標，座標是從小到大排序的；第三行有 N 個正整數代表樹的高度。同一行數字之間以空白間隔， $N \leq 1e5$ ， L 與樹高都不超過 $1e9$ 。

輸出：第一行輸出能被砍除之樹木數量，第二行輸出能被砍除之樹木中最高的高度。

範例輸入：

```
6 140
10 30 50 70 100 125
30 15 55 10 55 25
```

範例結果：

```
4
30
```

題目中有句話：無論砍樹的順序為何，最後能砍除的樹木是相同的。所以不斷找到滿足砍除條件的樹木，將它砍倒後移除，直到沒有樹木可以砍為止，就可以得到正確的答案，問題只是效率好不好。我們先看看天真的作法。

寫程式最重要的是知道每次要做甚麼事情，資料中存放的是甚麼，有何特性。如果我們用陣列來存這些樹，那麼立刻碰到一個問題：當樹被砍掉了之後怎麼辦？我們當然必須維護好陣列中是沒有被砍掉的樹，否則判斷就會出問題。第一個想法就是想要把砍倒的樹的資料從陣列中移除，那麼陣列中如何移除資料呢？只能把後面的往前搬，陣列最經不起插入與刪除的折騰，好吧，反正電腦累，程式碼倒是簡單。

下面的範例程式是這樣的想法寫出來的。先寫一個副程式從頭到尾檢查找到第一根可以砍的樹。這裡我們加了一個小技巧，在兩個邊界種上無限高的樹，這是常用的技巧，目的是省卻檢查邊界條件的麻煩。主程式中每次呼叫副程式，找到可以砍的樹之後就藉著搬移後面的資料來移除這筆資料。

```
// O(n^2), remove cut tree from array
#include <stdio.h>
#define N 100010
#define oo 1000000000
// return a removable tree; return -1 if none
int removable(int c[], int h[], int k) {
    for (int i=1; i<k-1; i++)
        if ((c[i]-h[i]>=c[i-1]) || (c[i]+h[i]<=c[i+1]))
            return i;
    return -1;
}

int main() {
    int n, l, c[N], h[N]; // center and height
    int total=0, high=0; // total removed and max height
    scanf("%d%d", &n, &l);
    // set left and right boundaries
```

```

c[0]=0, h[0]=oo;
c[n+1]=1, h[n+1]=oo;
for (int i=1;i<=n;i++)
    scanf("%d",&c[i]);
for (int i=1;i<=n;i++)
    scanf("%d",&h[i]);
n += 2; // two boundary
while (1) {
    int cut = removable(c, h, n); // the previous alive
    if (cut < 0) break;
    total++;
    if (h[cut] > high) high=h[cut];
    // remove cut from array
    for (int i=cut; i<n-1; i++)
        h[i] = h[i+1]; // overwrite by next
    for (int i=cut; i<n-1; i++)
        c[i] = c[i+1];
    n--;
}
printf("%d\n%d\n", total, high);
return 0;
}

```

複雜度如何？ $O(n^2)$ ，因為砍樹的順序剛好是從前往後，那麼總共搬移的資料量就會是從 1 加到 $n-1$ 。處理陣列資料被移除的方式除了真的把他移除之外，還有一種常用的方法是將他標記，這一題我們可以把砍除的樹標記為高度 0，這麼一來刪除就只花 $O(1)$ ，但是有一好沒兩好，我們在檢查左右存活的樹的時候，就必須往左往右尋找，因為旁邊的樹可能已經被砍除。以下是這樣寫出來的程式，很不幸，複雜度還是 $O(n^2)$ 。

```

// P_3_6, n^2 method, mark removed tree
#include <stdio.h>
#define N 100010
#define oo 1000000000

int main() {
    int n, l, c[N], h[N]; // center and height;
    int total=0, high=0; // total removed and max height
    scanf("%d%d",&n,&l);
    // set left and right boundaries
    c[0]=0, c[n+1]=1;
    h[0]=h[n+1]=oo;
    for (int i=1; i<=n; i++)
        scanf("%d", &c[i]);
    for (int i=1; i<=n; i++)
        scanf("%d", &h[i]);
    // repeat until no tree can be removed
    while (1) {
        int i;
        for (i=1; i<=n; i++) { // check each tree

```

```

        if (h[i]==0) continue; // already removed
        int prev=i-1, next=i+1;
        while (h[prev]==0) prev--; // previous remaining tree
        while (h[next]==0) next++; // next remaining tree
        if (c[i]-h[i]>=c[prev] || c[i]+h[i]<=c[next])
            break; // removable
    }
    if (i>n) break; // no tree can be removed
    total++;
    if (h[i] > high) high=h[i];
    h[i]=0;
}
printf("%d\n%d\n", total, high);
return 0;
}

```

這一題有人以為應該從矮的樹開始砍，但這顯然沒有甚麼幫助，因為高的樹可能比矮的樹先砍除。其實我們只要想想：目前不能被砍除的樹，什麼狀況會變成可以砍除呢？一定與他相鄰的（還存活的）樹被砍掉，才有可能空出多餘的空間。加油！解法就快要浮現了。

假設我們以一個佇列 Q 存放那些砍除的樹，一開始先檢查一次那些樹可以被砍除，把他們放入 Q ，然後我們一次將 Q 中的樹拿出來，檢查他左右相鄰的樹是否可以變成可以砍除，如果可以，就砍除並放入 Q ，這樣做直到 Q 中沒有樹為止。因為每一棵被砍除的樹都有考慮他的左右，所以不會漏砍。這是個正確的算法，但是這個過程中牽涉一個問題，就是必須很快的決定某棵樹的左右鄰居。趁這個機會我們簡單介紹一下串列鏈結 (linked list)。

串列鏈結 (Linked list)

串列是一個很重要的資料結構，顧名思義，他是將資料以鏈結的方式串成一列。他最重要的運用場合是當資料無法依序儲存在連續空間的時候，否則，用陣列會比串列更簡單有效率。但是陣列的缺點就是必須連續擺放，在某些應用時，這個限制變得不可能，例如檔案存在硬碟中，因為檔案會新增與刪除，所以將一個檔案放在連續的磁碟位置變得不可能。鏈結串列的概念是不必把資料連續擺放，只要對每一筆資料記載他的下一筆所在位置，有了這些資料，我們就可以一筆接一筆的把全部的資料追蹤一遍了。鏈結串列有單向與雙向，看應用而定，雙向的串列我們會儲存每一個資料的前一筆與下一筆鏈結，單向的只存下一筆。

通常鏈結串列在運用時多半搭配記憶體的配置 (memory allocation) 與指標 (pointer) 變數，但是在解題程式中，我們通常會避免這兩個東西，原因之一是指標很難除錯，另外一個原因是解題的程式通常都有辦法避掉指標，另外用偷懶的方式來做。

這份教材是針對考試與競賽的解題，所以我們就不對這個部份多做說明了，有興趣的讀者可以很容易在網路上或者資料結構的教科書中找到教學文件。在解題的場合，我們可以在陣列中構造所需的鏈結串列，用陣列的索引 (index) 來取代記憶體位置 (指標)。我們接著就看看串列如何用在 P-3-6。

在下面的範例程式中，我們定義了一個 struct，裡面存放一棵樹的資料，包括：位置、高度、前一棵的位置、下一棵的位置、以及是否還存活。這裡需要使用雙向鏈結，其中前一棵與下一棵的位置都是指在陣列中的索引值。如前所述，我們以一個佇列 Q 存放那些砍除的樹。函數 removable() 用來檢查第 i 棵樹是否可以被砍掉，如果可以，我們會將他從串列中移除，並且放入 Q 中。把一筆資料從串列中移除不需要像陣列一樣搬一大堆資料，只需要將左邊的 next 改成 i 的 next，把右邊的 pre 改成 i 的 pre，這樣在串列中沿著鏈結追蹤時就再也找不到 i 了，也就相當於從串列中移除了。

主程式一開始先做一次地毯式搜索，對每一個檢查每一個 i 呼叫一次 removable(i)，接著進入一個 while 迴圈，迴圈會一直做到 Q 必成空的為止。迴圈中每次從 Q 中拿出一棵樹，對他的左右會叫一次 removable()。這個程式的複雜度是多少？很顯然 removable() 是 $O(1)$ ，因為其中沒有迴圈也沒有遞迴。因為每棵樹最多只會被放入 Q 中一次，所以整體的複雜度是 $O(n)$ 。

```
// P_3_6b, topological sort and linked list
#include <queue>
#include <cstdio>
using namespace std;
#define N 100010
#define oo 1000000001
struct {
    int c, h; // center and height
    int pre, next; // linked list
    int alive;
} tree[N];
queue<int> Q; // queue for removed tree
// check if i is removable, if yes, remove it
void removable(int i) {
    if (!tree[i].alive) return;
    int s=tree[i].pre, t=tree[i].next;
    if (tree[i].c - tree[i].h >= tree[s].c \
        || tree[i].c+tree[i].h <= tree[t].c) {
        tree[i].alive = 0;
        Q.push(i); // insert into queue
        tree[s].next = t; //remove from linked list
        tree[t].pre = s;
    }
    return;
}

int main() {
```

```

int n,l,i;
int total=0, high=0;
scanf("%d%d",&n,&l);
// initial data
for (i=1; i<=n; i++)
    scanf("%d", &tree[i].c);
for (i=1; i<=n; i++)
    scanf("%d", &tree[i].h);
for (i=1; i<=n; i++) {
    tree[i].pre = i-1;
    tree[i].next = i+1;
    tree[i].alive = 1;
}
// two boundaries
tree[0].c = 0, tree[0].h = oo;
tree[n+1].c = l, tree[n+1].h = oo;
// initial check
for (i=1; i<=n; i++)
    removable(i);
while (!Q.empty()) {
    int v=Q.front();
    Q.pop();
    total++;
    high = max(high, tree[v].h);
    // check its previous and next
    removable(tree[v].pre);
    removable(tree[v].next);
}
printf("%d\n%d\n", total,high);
return 0;
}

```

上面這個程式的方法，其實類似於圖論演算法中計算 dag 的 topological sort 所用的方式，我們在後續介紹圖的章節中會再介紹。事實上，這一題還有更簡單的作法。

回到剛才講到的關鍵點：一棵目前不能被砍除的樹，只有在相鄰的樹被砍掉後才有可能變成可以砍除。假設我們從前往後掃描所有的樹，可以砍的就砍了，不能砍的暫且保留起來，那麼被保留的樹和時會變成可以被砍呢？因為他的左方（前方）不會變動（我們從左往右做），所以一定是右邊的樹被砍了之後才有可能。此外，那些被保留的樹，一定只有最後一棵需要考慮！因為其他的被保留樹的右邊都沒有動。好了，最後的問題是要怎麼存那些被保留的樹呢？我們一樣可以用鏈結串列，但是沒有必要，因為我們只要知道每一棵樹還存活的前一棵樹就可以，而且（重點是）我們只需要檢查最後一棵保留樹，也只需要從後面往前刪除。答案出來了，用堆疊就夠了。

以下是用堆疊寫的範例程式。需要留意的是，當一棵樹被砍除時，我們要用一個 while 迴圈對堆疊出口的樹做檢查，因為可能砍到一棵可能引發連鎖效應一路往前砍掉很多棵。複雜度 $O(n)$ ，因為每個樹最多只進入堆疊一次。請注意我們在堆疊中只放樹的 index，當然也可以寫成在堆疊中存放樹的位置與高度。

```

// P_3_6, O(n) using stack
#include <bits/stdc++.h>
using namespace std;
#define N 100010
#define oo 1000000000

int main() {
    int n,l,i, c[N],h[N];
    int total=0, high=0; // num of removable and max height
    stack<int> S; // the index of remaining trees so far
    scanf("%d%d",&n,&l);
    // oo trees at two boundaries
    c[0]=0;
    h[0]=oo;
    c[n+1]=l;
    h[n+1]=oo;
    for (i=1;i<=n;i++)
        scanf("%d", &c[i]);
    for (i=1;i<=n;i++)
        scanf("%d", &h[i]);
    S.push(0);
    for (i=1;i<=n;i++) { // scan from left to right
        // if i is removable
        if (c[i]-h[i]>=c[S.top()] || c[i]+h[i]<=c[i+1]) {
            total++;
            high = max(high, h[i]);
            // backward check remaining tree in stack
            while (c[S.top()+h[S.top()]<=c[i+1]) {
                total++; high = max(high, h[S.top()]);
                S.pop();
            }
        } else { // i is not removable
            S.push(i);
        }
    }
    printf("%d\n%d\n", total,high);
    return 0;
}

```

滑動視窗 (Sliding window)

接下來要介紹滑動視窗的技巧，這個名字可能不是很統一，基本上是以兩個指標維護一段區間，然後逐步將這區間從前往後（從左往右）移動。維護兩個指標這一類的方法也有人稱為雙指標法，但也包含兩個指標從兩端往中間移動，就像我們曾在介紹過，在排好序的序列中尋找兩數相加之和的問題時，可以用兩個指標逐步移動來做，會比連續的二分搜還好。也有人稱為爬行法，維護一個逐步右移的區段很像是毛毛蟲的爬行，前方先往前，後方再跟上，不斷的重複這個亦步亦趨的方式。反正名稱不重要，我們來看看一些例子。

下一個例題與 P-2-11 幾乎相同，但此處限制輸入的數字為正整數，原題正負不限，所以這一題比較簡單。原題必須以 set 來解超過 APCS 考試範圍，這一題則是滑動視窗的基本題型。

P-3-7. 正整數序列之最接近的區間和

輸入一個正整數序列 ($A[1], A[2], \dots, A[n]$)，另外給了一個非負整數 K ，請計算哪些連續區段的和最接近 K 而不超過 K ，以及這樣的區間有幾個。 n 不超過 20 萬，輸入數字與 K 皆不超過 10 億。

Time limit: 1 秒

輸入格式：第一行是 n 與 K ，第二行 n 個整數是 $A[i]$ ，同行數字以空白間隔。

輸出格式：第一行輸出最接近 K 但不超過 K 的和，第二行輸出這樣的區間有幾個。

範例輸入：

```
5 10
4 3 1 7 4
```

範例輸出：

```
8
2
```

說明：(4, 3, 1) 與 (1, 7) 兩個區間的和都是 8。

假設一開始我們先固定左端在 0 的位置，從 0 開始一直移動右端以尋找最好的右端，也就是區段的和不超過 K 的最大，因為數字都是正的，右端越往右移，區段和就越大，所以當移動到不能再移時 (否則就會超過 K)，我們就找到了「以 0 為左端的最好區間」。接著我們將這個區段 (視窗) 逐步右移，每次先將右端往右移動一格，這時總和可能會超過 K ，所以我們要移動左端直到滿足區段和不超過 K 為止。每次右端移動一格，就會找到以此為右端的最佳解，所以當視窗向右滑到底時，我們就找出了最佳解。範例程式如下，以可以用 P_2_11 的程式來解做為對照，不過要稍微修改一下。

```
// P3-7, sum of subarray, positive
#include <bits/stdc++.h>
using namespace std;
#define N 1000010
int a[N];

int main() {
    int n, k;
    long long total=0;
```



```

scanf("%d%d",&n,&k);
for (int i=0;i<n;i++) {
    scanf("%d", &a[i]);
    total+=a[i];
}
int left, right,ans=0, sum=0, num=0;
// sum is for range [j,i]
// move right one by one
for (left=0,right=0;right<n;right++) {
    sum+=a[right];
    // move left until sum<=k
    while (sum>k) {
        sum-=a[left];
        left++;
    }
    // check answer
    if (sum>ans) {
        ans=sum;
        num=1;
    }
    else if (sum==ans) num++;
}
printf("%d\n%d\n",ans,num);
fprintf(stderr,"total=%lld, k=%d; %d %d\n",total,k,ans,num);
return 0;
}

```

上面一題的視窗大小是會變動的，其實比較像毛毛蟲，不像窗戶。下面一題我們來個固定長度的窗戶。

P-3-8. 固定長度區間的最大區段差

對於序列的一個連續區段來說，區段差是指區段內的最大值減去區段內的最小值。有 N 個非負整數組成的序列 seq ，請計算在所有長度為 L 的連續區段中，最大的區段差為何。

Time limit: 1 秒

輸入格式：第一行是 N 與 L ，第二行是序列內容，相鄰數字間以空白隔開。 $L \leq N \leq 2e5$ ，數字不超過 $1e9$ 。

輸出：輸出所求的最大區間差。

範例輸入：

```

9 4
1 4 3 6 9 8 5 7 1

```

範例結果：

7

說明：(8,5,7,1)的長度是 4，區段差是 7。

連續的 L 個格子是一個窗戶，長度 N 的序列有 $N-L+1$ 個可能的窗戶位置，對每一個位置，把窗戶內的最大值減去最小值，可以得到這個區段的區段差，所有 $N-L+1$ 個區段差中找到最大的。直接的天真做法當然不行，每個區段跑一遍找最大最小需要 $O(L)$ ， $N-L+1$ 個區段就是 $O(L(N-L+1))$ ， $L=N/2$ 時，就是 $O(N^2)$ 。

這個題目解法的主要想法是紀錄區段內的最大值與最小值，當區段往右移動時，更新最大與最小值。乍看之下，更新最大最小值很容易就可以做到，因為只移進來一個值，移出去一個值。可是仔細想想，如果移出去的是最大值，那最大值會變成多少呢？可能會變也可能不變，這麼想下去覺得問題沒有那麼簡單。

前一章學過的 `multiset` 是否有用呢？因為 `set` 可以簡單的找到最大與最小，移進移出也都可以操作，所以確實以 `multiset` 可以來幫忙，時間複雜度是 $O(N \log(N))$ 。不過這裡要介紹的是 $O(N)$ 的解法，用的資料結構是 `deque`，也就是雙向佇列。

我們要不斷的找出窗戶內的最大值與最小值，我們先看最大值的找法，最小值的做法完全類似。其實這一題跟例題「P-3-4 最接近的高人」有點像，有個相同的單調性觀察：「當窗戶的右端推到 i 時，在 i 之前所有不大於 `seq[i]` 的元素，對未來找最大值都是沒有用的。」因此，我們可以刪除那些沒用的資料，只要維護好窗戶內的一個遞減子序列，與 P-3-4 不同處在於，因為窗戶會往右移動，資料有可能從窗戶左端離開，因此我們不能用堆疊來做，要用 `deque` 來做，以下是範例程式。

找最小值的做法相同，只要改變大小關係就可以了，所以我們用兩個 `deque`：`max_q` 與 `min_q`。函數 `put_max(i)` 處理將 `seq[i]` 從窗戶右端放入 `max_q` 時的動作：先從 `max_q` 的後端移除所有不大於它的成員（它是個遞減序列），然後把自己推進 `max_q` 內。在主程式中，跑一個迴圈讓窗戶的右端 i 逐步往右推，每次執行 `put_max()` 與 `put_min()`，另外檢查兩個 `deque` 的前端元素是否已經離開窗戶左端。處理完畢後 `max_q.front()` 就是窗戶內的最大值，`min_q.front()` 就是最小值。留意因為比較短的區段差不會比較大，所以初始時我們窗戶右端從 0 開始跑，此時窗戶寬度小於 L 但並不影響答案。

```
// P3-8, O(N) using deque
#include <bits/stdc++.h>
using namespace std;
#define MAXN 200010
```

```

int seq[MAXN];
deque<int> max_q, min_q; // index of max and min queue
// put seq[i] from back of max_q
void put_max(int i) {
    // remove useless
    while (!max_q.empty() && seq[max_q.back()]<=seq[i])
        max_q.pop_back();
    max_q.push_back(i);
}
// put seq[i] from back of min_q
void put_min(int i) {
    // remove useless
    while (!min_q.empty() && seq[min_q.back()]>=seq[i])
        min_q.pop_back();
    min_q.push_back(i);
}

int main() {
    int n, L, i, j, max_diff=0;
    scanf("%d%d\n", &n, &L);
    for (i=0; i<n; i++) {
        scanf("%d", &seq[i]);
    }
    put_max(0);
    put_min(0);
    for (i=1; i<n; i++) {
        // put the ith element into max_queue
        if (max_q.front()<=i-L) // out-of-range
            max_q.pop_front(); // its index
        put_max(i);
        // put the ith element into min_queue
        if (min_q.front()<=i-L) // out of range
            min_q.pop_front();
        put_min(i);
        // the diff of this subarray
        int diff=seq[max_q.front()]-seq[min_q.front()];
        max_diff=max(max_diff, diff);
    }
    printf("%d\n", max_diff);
    return 0;
}

```

前面的題目都是跟序列中數字的大小有關係，接下來看有數字但沒有大小的題目，一共有四個類似的題目，我們示範兩題，另外兩題當做習題。

P-3-9. 最多色彩帶

有一條細長的彩帶，彩帶區分成 n 格，每一格的長度都是 1，每一格都有一個顏色，相鄰可能同色。給定長度限制 L ，請找出此彩帶長度 L 的連續區段最多可能有多少種顏色。

Time limit: 1 秒

輸入格式：第一行是兩個正整數 n 和 L ，滿足 $2 \leq L \leq n \leq 2 \times 10^5$ ；第二行有 n 個以空白間隔的數字，依序代表彩帶從左到右每一格的顏色編號，顏色編號為小於 n 的非負整數。

輸出：長度 L 的彩帶區段最多有多少種顏色。

範例輸入：

```
10 5
4 6 1 4 0 6 8 0 5 6
```

範例結果：

```
5
```

說明：區間 $[3, 7]$ 有 5 色。

這是一個簡單的題目，把一個固定長度 L 的視窗，從左到右逐步滑動過去，找出視窗內的顏色總數就行了。所以重點只有一個：如何找出是窗內的顏色數。因為顏色是從 0 開始連續編號，我們只要準備一個表格 cnt ，以 $\text{cnt}[i]$ 紀錄顏色 i 的出現次數。每次進入一個顏色，就將該顏色計數器加一；移出一個顏色就將顏色計數器減一。那怎麼知道窗內的顏色數呢？不能每次都掃描表格，因為太浪費時間，我們可以只關注顏色數的變動。我們可以用一個變數記錄目前的顏色數，若 $\text{cnt}[i]++$ 之後變成 1，那就知道顏色數多了一種；相同的，如果 $\text{cnt}[i]$ 減 1 之後變成 0 就是少了一色，其他狀況的顏色數量沒變化。

```
// P_3_9, sliding window, max color of fixed length
#include<bits/stdc++.h>
using namespace std;
#define N 200010
int seq[N], cnt[N]={0}; // counter of color i

int main() {
    int n,L,i;
    scanf("%d%d",&n,&L);
    int n_color=0;
    for (i=0;i<n;i++)
        scanf("%d",&seq[i]);
    // initial window
    for (i=0;i<L;i++) {
        int color=seq[i];
        cnt[color]++;
        if (cnt[color]==1) n_color++;
    }
    int ans=n_color, left;
```

```

// sliding window, seq[i] in, seq[left] out
for (left=0,i=L; i<n; i++,left++) {
    int color=seq[i];
    cnt[color]++;
    if (cnt[color]==1) n_color++;
    color=seq[left];
    cnt[color]--;
    if (cnt[color]==0) n_color--;
    ans=max(ans,n_color);
}
printf("%d\n",ans);
return 0;
}

```

下面一題很類似，但多了一些要處理的問題。

P-3-10. 全彩彩帶（需離散化或字典）（@@）

有一條細長的彩帶，彩帶區分成 n 格，每一格的長度都是 1，每一格都有一個顏色，相鄰可能同色。如果一段彩帶中的顏色包含整段彩帶的所有顏色，則稱為「全彩彩帶」。請計算出最短的全彩彩帶長度。

Time limit: 1 秒

輸入格式：第一行為整數 n ，第二行有 n 個以空白間隔的非負整數，依序代表彩帶從左到右每一格的顏色編號， $n \leq 2e5$ ，顏色編號不超過 $1e9$ 。

輸出：最短的全彩彩帶長度。

範例輸入：

```

10
6 4 1 6 0 4 5 0 7 4

```

範例結果：

```

7

```

說明：彩帶共有 $\{0, 1, 4, 5, 6, 7\}$ 六色區，區間 $[3, 9]$ 是最短的包含六色區段。

基本上我們打算維持住一個視窗，視窗裡面包含所有的色彩，每次當視窗往右堆一格的時候，檢查左端是否可以往右移以便縮減寬度，當視窗推到底之後，我們就已經檢查過所有可能的右端點，在過程中找到最窄的視窗寬度，就是答案。

何時左端可以縮減呢？答案很明顯，如果左端點顏色在是窗內出現超過一次，就可以丟棄，否則，不能右移。另外一個問題是顏色的編號範圍，在前一題 P-3-9 時，顏色的編號不大於 n ，所以我們可以很容易的開一個表格當每個顏色的計數器，但是在這一題，顏色的編號可能達到 10 億，想要開一個 10 億的表格是會碰上麻煩的。雖然編號可能高達 10 億，但是資料最多就只有 20 萬個，顏色當然最多也只有 20 萬種，因此，如果我們可以將顏色的編號先修改為 0 開始的連續編號，就可以用上一題的表格方式來處理計數器了。在第二章的 P-2-2 與 P-2-2C 中我們示範了如何用 sort 或 map 做離散化，在這裡就可以派上用場了。下面的範例程式是以 sort 以及自己寫二分搜來做離散化的寫法，副程式 `c_id()` 就是自己寫的二分搜來查詢 `color` 在 `dic[]` 中的位置。在 35~36 行我們把所有的 `seq[]` 都換成 `rank` 之後就完成了離散化的工作。接著進行滑動視窗來找答案。以兩個變數 `left` 與 `right` 來維持至目前的視窗範圍，`right` 每次加 1，`left` 則只要顏色還有重複就盡可能縮減。要注意的是，為了簡化程式碼，我們並不保證一開始的視窗是包含所有的色彩的，但是在第 49 行加了一個 `if` 來確保當全部的色彩都在時，才會記錄答案。如果一開始要先找到出於全彩視窗也可以，但可能要多寫幾行。這個範例的程式碼雖然有一點長（其實也不算太長），但是方法的正確性不難理解，重點在於左端的顏色如果在是窗內僅出現一次，那就絕對不能右移。

```

00 // P_3_10 shortest all-color range, only basic instruction
01 #include<bits/stdc++.h>
02 using namespace std;
03 #define N 1000010
04 int seq[N], cnt[N]={0}; // counter of color
05 int dic[N]; // dictionary, map color to 0~m-1
06 // binary search to find id of color
07 int c_id(int color, int nc) {
08     if (color<=dic[0]) return 0;
09     int p=0;
10     for (int jump=nc/2; jump>0; jump>>=1) {
11         while (p+jump<nc && dic[p+jump]<color)
12             p+=jump;
13     }
14     return p+1;
15 }
16
17 int main() {
18     int n;
19     scanf("%d",&n);
20     for (int i=0;i<n;i++)
21         scanf("%d",&seq[i]);
22     // discretization
23     for (int i=0;i<n;i++)
24         dic[i]=seq[i];
25     sort(dic, dic+n);
26     // remove duplicate color
27     int n_color=1; // num of color
28     for (int i=1;i<n;i++) { // not checking dic[0]

```

```

29     if (dic[i]!=dic[i-1]) {
30         dic[n_color]=dic[i];
31         n_color++;
32     }
33 }
34 // replace color with its rank
35 for (int i=0;i<n;i++)
36     seq[i]=c_id(seq[i],n_color);
37 // end of discretization, start sliding window
38 // w_color = num of color in window [left, right-1]
39 int left=0, right=0, w_color=0, shortest=n;
40 while (right<n) {
41     cnt[seq[right]]++;
42     if (cnt[seq[right]]==1) w_color++;
43     right++;
44     // shrink left until left color appear only once
45     while (cnt[seq[left]]>1) {
46         cnt[seq[left]]--;
47         left++;
48     }
49     if (w_color==n_color)
50         shortest=min(shortest, right-left);
51 }
52 printf("%d\n",shortest);
53 return 0;
54 }
55

```

在第二章我們曾經揭露出以 STL 中的 map 來做離散化的寫法，底下是這一題用 map 的範例程式。這裡再強調一次，以 APCS 來說，應該不至於會考出非用特殊資料結構 (如 map) 才能解的題目，但是很多題目如果使用這些資料結構可以簡化我們的程式，在考試與比賽時，程式碼的長短對犯錯機率與 debug 影響很大，所以要不要學，要看個人狀況而定，程度已經夠了，就多學一點好用的工具，如果對基本的程式還很生疏，就練習熟練後再說。

```

// P_3_10 shortest all-color range, using map
#include<bits/stdc++.h>
using namespace std;
#define N 1000010
int seq[N], cnt[N]={0}; // counter of color
map<int,int> dic; // dictionary

int main() {
    int n;
    scanf("%d",&n);
    // input and discretization
    for (int i=0;i<n;i++) {
        scanf("%d",&seq[i]);
        dic[seq[i]]=0; // insert if not existing
    }
}

```

```

}
int n_color=0; // num of color
for (auto &p:dic) p.second=n_color++;
// find initial window of all color
int left=0, right=0, w_color=0, shortest=n;
// w_color = num of color in window [left, right-1]
// sliding window
while (right<n) {
    int color=dic[seq[right]];
    cnt[color]++;
    if (cnt[color]==1) w_color++;
    right++;
    // shrink left until left color appear only once
    while (1) {
        color=dic[seq[left]];
        if (cnt[color]==1) break;
        cnt[color]--;
        left++;
    }
    if (w_color==n_color)
        shortest=min(shortest, right-left);
}
printf("%d\n",shortest);
return 0;
}

```

這一題因為顏色數字並沒有大小關係，所以其實順序不重要，離散化的過程可以用 `unordered_map` 取代 `map`，不過這份教材並沒有打算介紹那個資料結構，所以就不多說了，但範例程式中有附，有興趣的人可以自己參考。

上面的寫法，如果不計離散化，時間複雜度都只有 $O(n)$ ，這一題應該已經完美了，但以下還要提出一個比較慢的方法！慢且(台語)！學完快的為什麼還要學慢的呢？解題不是重點，技術才是重點，因為題目千變萬化，只有學到了技術才有用。下面這個方法饒富趣味，而且後面的章節我們也會碰到。

對於一個固定的輸入序列來說，令 $f(L)$ 是長度 L 的連續區段可以擁有的最多顏色數。那麼，P-3-9 就是對輸入的 L 計算 $f(L)$ 的值，而 P-3-10 則是要計算滿足 $f(L) \geq nc$ 的最小 L 值，其中 nc 是總共的顏色數。函數 $f()$ 明顯有單調性，因為長度更長的情況下，最多擁有的顏色數只會增加或不變，不會變少。因此，如果把 P-3-9 的程式寫成函數，然後不斷的呼叫 $f()$ 去計算 $f(1)$, $f(2)$, ..., 直到第一個回傳值不小於 nc 時，我們就找到了 P-3-10 的解。且慢，既然 $f()$ 具單調性，線性搜尋就太遜了，我們當然可以用二分搜。以下是這樣的想法寫出的程式，時間複雜度是多少？如果離散化不計，P-3-9 是 $O(n)$ ，長度範圍是 $1 \sim n$ ，二分搜會呼叫 $O(\log(n))$ 次，所以是 $O(n \log(n))$ 。

```

// P_3_10 shortest all-color range, repeatedly call P-3-9

```



```

// using binary search to find shortest
#include<bits/stdc++.h>
using namespace std;
#define N 1000010
int seq[N];
int dic[N]; // dictionary, map color to 0~m-1
// max num of color of window size w
int w_col(int w, int n) {
    vector<int> cnt(n, 0); // initial counter
    int i, j, nc=0;
    for (i=0;i<w;i++) {
        if (++cnt[seq[i]] ==1)
            nc++;
    }
    int maxc=nc;
    // insert i, delete j
    for (j=0, i=w;i<n;i++,j++) {
        if (++cnt[seq[i]] ==1)
            nc++;
        if (--cnt[seq[j]] ==0)
            nc--;
        maxc=max(maxc,nc);
    }
    return maxc;
}

int main() {
    int n;
    scanf("%d",&n);
    for (int i=0;i<n;i++)
        scanf("%d",&seq[i]);
    // discretization
    for (int i=0;i<n;i++)
        dic[i]=seq[i];
    sort(dic, dic+n);
    // remove duplicate color
    int n_color=1; // num of color
    for (int i=1;i<n;i++) { // not checking dic[0]
        if (dic[i]!=dic[i-1]) {
            dic[n_color]=dic[i];
            n_color++;
        }
    }
    // replace color with its rank
    for (int i=0;i<n;i++)
        seq[i]=lower_bound(dic,dic+n_color,seq[i])-dic;
    // end of discretization
    if (n_color==1) {printf("1\n"); return 0;}
    // using binary search to find the longest invalid length
    int length=1;
    for (int jump=n/2; jump>0; jump>>=1) {
        while (w_col(length+jump,n)<n_color)
            length+=jump;
    }
    printf("%d\n",length+1);
}

```

```

    return 0;
}

```

接下來兩題與前兩題類似，我們當做習題。

Q-3-11. 最長的相異色彩帶

有一條細長的彩帶，彩帶區分成 n 格，每一格的長度都是 1，每一格都有一個顏色，相鄰可能同色。如果一段彩帶其中的每一格顏色皆相異，則稱為「相異色彩帶」。請計算最長的相異色彩帶的長度。

Time limit: 1 秒

輸入格式：第一行為整數 n ，滿足 $n \leq 2 \times 10^5$ ；第二行有 n 個以空白間隔的數字，依序代表彩帶從左到右每一格的顏色編號，顏色編號是不超過 n 的非負整數。

輸出：最長的相異色彩帶的長度。

範例輸入：

10

6 4 1 6 0 4 5 0 7 4

範例結果：

5

說明：區間 $[3, 7]$ 的顏色 $(1, 6, 0, 4, 5)$ 皆不相同。

Q-3-12. 完美彩帶 (APCS201906)

有一條細長的彩帶，總共有 m 種不同的顏色，彩帶區分成 n 格，每一格的長度都是 1，每一格都有一個顏色，相鄰可能同色。長度為 m 的連續區段且各種顏色都各出現一次，則稱為「完美彩帶」。請找出總共有多少段可能的完美彩帶。請注意，兩段完美彩帶之間可能重疊。

Time limit: 1 秒

輸入格式：第一行為整數 m 和 n ，滿足 $2 \leq m \leq n \leq 2 \times 10^5$ ；第二行有 n 個以空白間隔的數字，依序代表彩帶從左到右每一格的顏色編號，顏色編號是不超過 10^9 的非負整數，每一筆測試資料的顏色數量必定恰好為 m 。

輸出：有多少段完美彩帶。

範例輸入：

```
4 10
1 4 1 7 6 4 4 6 1 7
```

範例結果：

3

說明：區間[2, 5]是一段完美彩帶，因為顏色 4、1、7、6 剛好各出現一次，此外，區間[3, 6]與[7, 10]也都是完美彩帶，所以總共有三段可能的完美彩帶。

下面的習題與 P-3-8 類似。

Q-3-13. X 差值範圍內的最大 Y 差值

輸入平面上 N 個點的座標 $(x[i], y[i])$ 以及一個正整數 L ，計算並輸出

$$\max_{1 \leq i < j \leq N} \{|y[i] - y[j]| : |x[i] - x[j]| \leq L\}.$$

Time limit: 1 秒

輸入格式：第一行是 N 與 L ，第二行各點的 x 座標，第三行依序是對應點的 y 座標，相鄰數字間以空白隔開。 $N \leq 2e5$ ，座標絕對值不超過 $1e9$ 。

輸出：輸出所求的最大差值。

範例輸入：

```
10 3
4 1 2 -10 3 5 6 9 7 8
6 1 4 10 3 9 8 1 5 7
```

範例結果：

7

說明： x 距離 3 以內的最大 y 差值是 $(6, 8)$ 與 $(9, 1)$ ， y 值差 7。

下面這個習題需要一點幾何知識，別擔心，國中就學過了。

Q-3-14. 線性函數 (00)

有 N 線性函數 $f_i(x) = a_i x + b_i$, $1 \leq i \leq N$ 。定義 $F(x) = \max_i f_i(x)$ 。輸入 $c[i]$, $1 \leq i \leq m$, 請計算 $\sum_{i=1}^m F(c[i])$ 。

Time limit: 1 秒

輸入格式：第一行是 N 與 m 。接下來有 N 行，依序每行兩個整數 a_i 與 b_i ，最後一行有 m 個整數 $c[1], c[2], \dots, c[m]$ 。每一行的相鄰數字間以空白隔開。 $N \leq 1e5$, $m \leq 5e4$, 輸入整數絕對值不超過 $1e8$, 答案絕對值不超過 $1e15$ 。

輸出：計算結果。

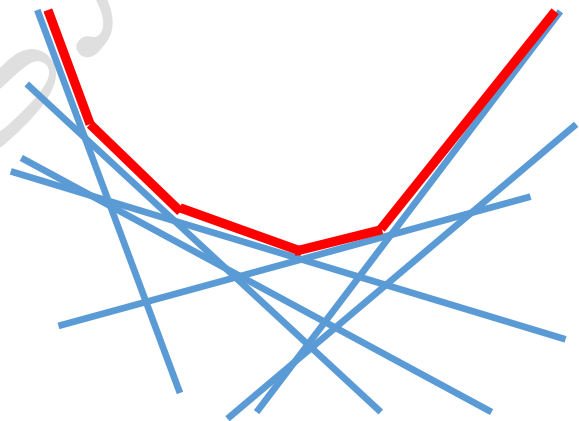
範例輸入：

```
4 5
-1 0
1 0
-2 -3
2 -3
4 -5 -1 0 2
```

範例結果：

```
15
```

提示：給 N 個線性函數 (一次函數)，另外定義 F 是這些函數的最大值，現在給 m 個 x 值，要計算 F 在這些點的函數值總和，也就是對每一個 x 值，要計算這些線性函數的最大值。直接的做法全部計算取最大，總共需要 $O(mn)$ ，效率不佳。請看右圖。藍線是這些線性函數，紅線就是 F ，他必定會形成一個 (向下) 凸的形狀，而且是一段一段的，每一段都是原來的某個藍線 (函數)，如果我們可以



找出這個紅線，計算這些函數值就沒問題了。我們需要一個觀察：若 f 與 g 是兩個線性函數， f 的斜率小於 g ，那麼，在兩線的交點以前是 f 大，以後是 g 大。我們可以將這些直線根據斜率排序，然後逐一將直線加入 F 。這一題另外也有分治的演算法可以做，在後續章節中會再出現。

4. 貪心演算法與掃描線演算法